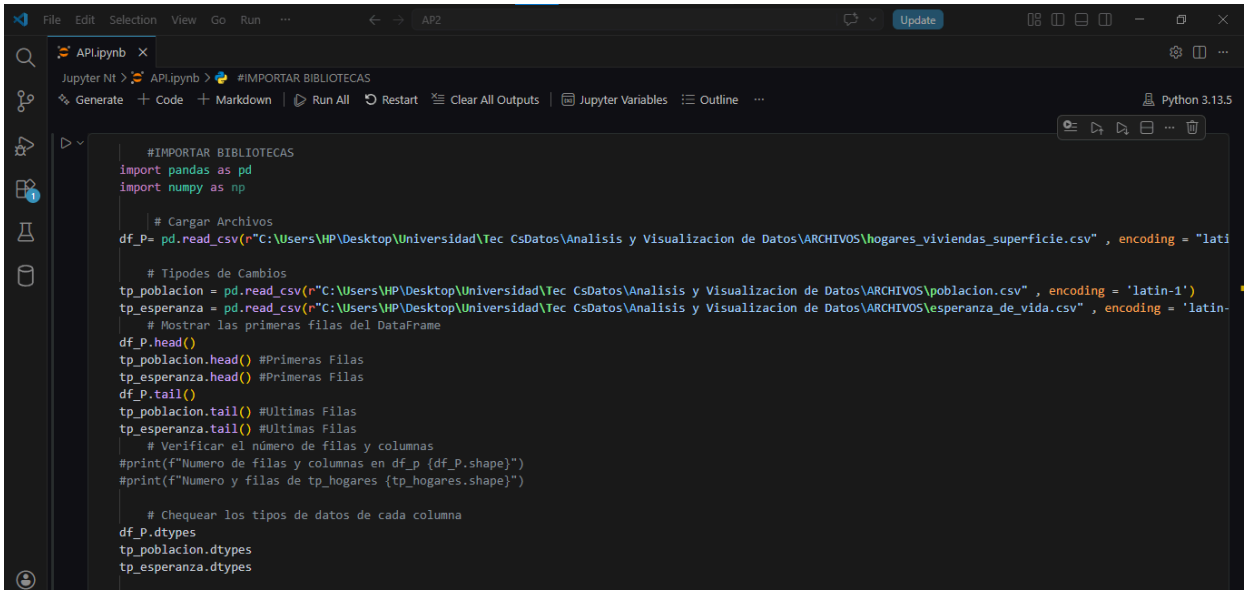
MANIPULACIÓN DE DATOS

Lo primero fue decidir que tipo de archivo utilice donde son : población , esperenza_de_vida y hogar_viviendas_superficie.

Lo que vemos acá es:

1. **Importación** de bibliotecas (pandas y numpy) con su respectivo alias.
2. **Cargar** el archivo central con el que vamos a trabajar (hogares_viviendas_superficie) , se carga con csv y latin -1 para que se leyera correctamente
3. **Luego** cargar los otros archivos correspondientes (población y esperanza)
4. **Comenzamos** a chequear los Data Frame:
    a. head → primeras filas
    b. tail → últimas filas
    c. dtypes → tipos de datos



```
#IMPORTAR BIBLIOTECAS
import pandas as pd
import numpy as np

# Cargar Archivos
df_P= pd.read_csv(r"C:\Users\HP\Desktop\Universidad\Tec CsDatos\Analisis y Visualizacion de Datos\ARCHIVOS\hogares_viviendas_superficie.csv" , encoding = "lati

# Tipodes de Cambios
tp_poblacion = pd.read_csv(r"C:\Users\HP\Desktop\Universidad\Tec CsDatos\Analisis y Visualizacion de Datos\ARCHIVOS\poblacion.csv" , encoding = 'latin-1')
tp_esperanza = pd.read_csv(r"C:\Users\HP\Desktop\Universidad\Tec CsDatos\Analisis y Visualizacion de Datos\ARCHIVOS\esperanza_de_vida.csv" , encoding = 'latin-
# Mostrar las primeras filas del DataFrame
df_P.head()
tp_poblacion.head() #Primeras Filas
tp_esperanza.head() #Primeras Filas
df_P.tail()
tp_poblacion.tail() #Ultimas Filas
tp_esperanza.tail() #Ultimas Filas
# Verificar el número de filas y columnas
#print(f"Numero de filas y columnas en df_p {df_P.shape}")
#print(f"Numero y filas de tp_hogares {tp_hogares.shape}")

# Chequear los tipos de datos de cada columna
df_P.dtypes
tp_poblacion.dtypes
tp_esperanza.dtypes
```

En esta imagen comenzamos con filtración , integracion y limpieza :

1. **Eliminación** de la columna ‘provincia_id’ con la función drop.
2. **Ordenar** con sort_values para ordenarlo alfabéticamente y decreciente
3. **Concatenamos** con merge los archivos tp_poblacion y tp_esperanza uniendolos a partir de las variables “anio” , “provincia” con how = ‘outer’ qué quiere decir registrar todos los datos por mas que no coincidan.
4. **Eliminamos** dos columnas que solo mostraba “NaN” con drop. Esto lo veo al verificar dicha concatenación se haya hecho correctamente
5. **Vuelvo** a concatenar los Data Frame principal y la anterior concatenación (tc). La palabra que se busca para la coincidencia es ‘provincia’
6. Luego corrijo el nombre de la columna ‘anio’ con rename

```python
## FILTROS de DATOS , INTEGRACION Y LIMPIEZA ##
df_P_fil = df_P.drop(columns = ['provincia_id' ]) # Eliminar columnas

df_P_fil = df_P_fil.sort_values(by = ['hogares' , 'viviendas_particulares'] , ascending = False) # Ordenar por hogares y viviendas particulares


tc = pd.merge(tp_poblacion , tp_esperanza , left_on = ['anio' , 'provincia'] , right_on = ['anio' , 'provincia'] , how = 'outer' ) # Concatenar los DataFrames
tc = tc.drop(columns = ['mujeres' , 'varones']) # Eliminar columnas

df_tt = pd.merge(tc , df_P_fil , on = ['provincia'] , how = 'outer' ) # Utilice merge porque las columnas coinciden

df_tt = df_tt.rename(columns = {'anio' : 'año'}) #Renombrar la columna

df_tt = df_tt[df_tt["provincia"].isin([ "Mendoza" , "San Juan" , "San Luis"])] # Filtrado
df_tt.isnull().sum() # Verificar valores nulos
df_tt['provincia'].unique() # Verificar valores únicos
```

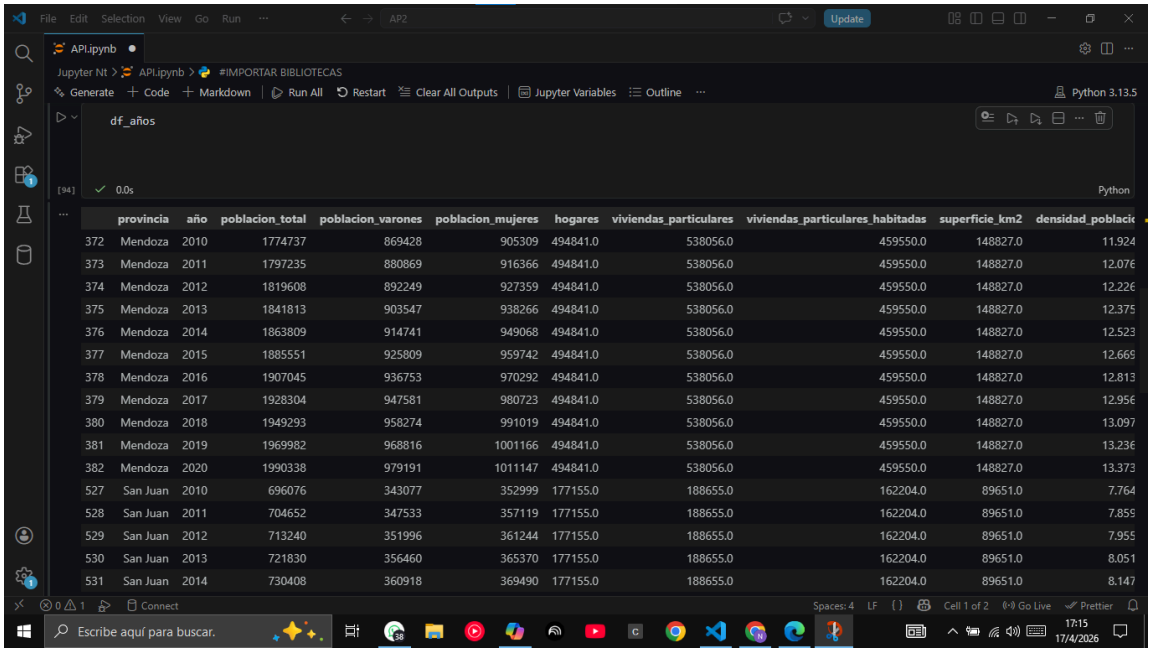
7. Luego de verificar que se haya el paso anterior correctamente , toca **FILTRAR** los datos donde decidí hacerlo por provincia San Juan y Mendoza con la función isin
8. Seguimos con la limpieza , viendo si existen valores nulos con isnull() y a su vez los sume con sum(). Donde salieron el total de número de 0
9. Y por último verificamos los valores únicos y que no haya ningún error com .unique()

Seguimo con el Análisis:

```python
## ANALISIS DESCRIPTIVOS ##
df_tt['hogares'].describe() # Calcular Estadistica Descriptiva

df_tt['densidad_poblacional'] = df_tt['poblacion_total'] / df_tt['superficie_km2'] # Crear nueva variable densidad poblacional

df_años = df_tt[df_tt['año'] <= 2020] # Filtrar por año

df_años['densidad_poblacional'].describe() # Calcular estadistica descriptiva de la nueva variable']

df_años
```

	provincia	año	poblacion_total	poblacion_varones	poblacion_mujeres	hogares	viviendas_particulares	viviendas_particulares_habitadas	superficie_km2	densidad_poblac
372	Mendoza	2010	1774737	869428	905309	494841.0	538056.0	459550.0	148827.0	11.9

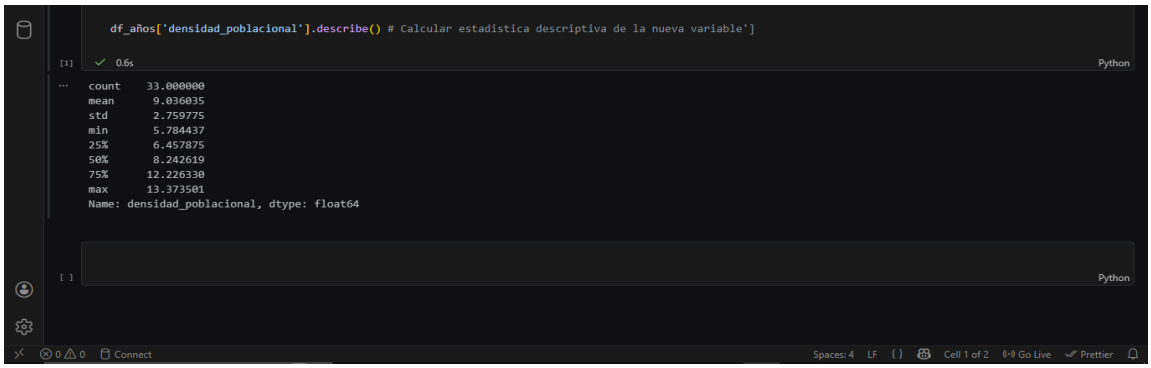
1. **Comenzamos** con la funciones .describe() en la columna hogares : aca te da información de estadistica basica (media, desviación, min, max)
2. **Generamos** una nueva variable o columna sobre la densidad de la población a partir de la división de las variables [poblacion_total] con [superficie_km2].
3. **Utilizamos** una filtración en la columna de años para que las muestras sean del año 2020 hacia abajo ; donde ahora el Data Frame se llama df_ años.
   Y aca esta el resultado de la tabla con dicho filtro que se observa la nueva columna “densidad_poblacional”



```
df_años
```

	provincia	año	poblacion_total	poblacion_varones	poblacion_mujeres	hogares	viviendas_particulares	viviendas_particulares_habitadas	superficie_km2	densidad_poblaci
372	Mendoza	2010	1774737	869428	905309	494841.0	538056.0	459550.0	148827.0	11.924
373	Mendoza	2011	1797235	880869	916366	494841.0	538056.0	459550.0	148827.0	12.076
374	Mendoza	2012	1819608	892249	927359	494841.0	538056.0	459550.0	148827.0	12.226
375	Mendoza	2013	1841813	903547	938266	494841.0	538056.0	459550.0	148827.0	12.375
376	Mendoza	2014	1863809	914741	949068	494841.0	538056.0	459550.0	148827.0	12.523
377	Mendoza	2015	1885551	925809	959742	494841.0	538056.0	459550.0	148827.0	12.669
378	Mendoza	2016	1907045	936753	970292	494841.0	538056.0	459550.0	148827.0	12.813
379	Mendoza	2017	1928304	947581	980723	494841.0	538056.0	459550.0	148827.0	12.956
380	Mendoza	2018	1949293	958274	991019	494841.0	538056.0	459550.0	148827.0	13.097
381	Mendoza	2019	1969982	968816	1001166	494841.0	538056.0	459550.0	148827.0	13.236
382	Mendoza	2020	1990338	979191	1011147	494841.0	538056.0	459550.0	148827.0	13.373
527	San Juan	2010	696076	343077	352999	177155.0	188655.0	162204.0	89651.0	7.764
528	San Juan	2011	704652	347533	357119	177155.0	188655.0	162204.0	89651.0	7.859
529	San Juan	2012	713240	351996	361244	177155.0	188655.0	162204.0	89651.0	7.955
530	San Juan	2013	721830	356460	365370	177155.0	188655.0	162204.0	89651.0	8.051
531	San Juan	2014	730408	360918	369490	177155.0	188655.0	162204.0	89651.0	8.147

Por último tenemos el resultado de **“df_años[‘densidad_poblacional].describe()”** donde observamos que la cantidad contada es de 33,00 con una media de 9.03 y una moderada dispersión de 2,750. Luego vemos que el rango mínimo (5,78) y máximo (13,37) hay una gran distancia entre ambos rangos (7,59).
Luego tenemos en el medio los percentiles indicando que la mayoría de la densidad de la población está dispersa por debajo del 75% queriendo decir que están entre el 6,45 y 12,22. Y que solo el 25% sobrepasa el umbral del 12,22
Por último y no menos importante, se observa que no hay outliers extremos (ningún valor desproporcionado) pero si una diferencia clara entre entre una provincia más densa que otra



```
df_años['densidad_poblacional'].describe() # Calcular estadistica descriptiva de la nueva variable']
```

```
count    33.000000
mean      9.036035
std       2.759775
min       5.784437
25%       6.457875
50%       8.242619
75%      12.226330
max      13.373501
Name: densidad_poblacional, dtype: float64
```

Debo aclarar que cada vez que generaba un cambio en el Data Frame lo evaluaba constantemente pero lo terminaba borrando dicha evaluación por solo instinto y eso hizo que no quede registrado todas las verificaciones que produje.
Luego lo que me costó más fue la concatenación entre las bases de datos ya que no entendía del todo la diferencia de cada función (merge y concat). Y luego al usar merge las denominaciones correctas para un correcto Data Frame.